Gerenciamento de Cidades

Ver | Gastar pelo menos 20 minutos na atividade | Passar pela atividade até o fim

Fecha: domingo, 14 dez. 2025, 23:59

Nessa primeira aula prática, iniciamos um projeto do tipo CRUD. Um CRUD é um aplicativo que permite criar (*create*), ler (*read*), alterar (*update*) e excluir (*delete*) dados. Como referência, usamos dados de cidades. Uma cidade está associada a um único estado. Portanto, para criar uma cidade precisamos informar o nome da cidade e o estado onde ela se encontra.

Editar lição | Editar o conteúdo desta página | Avaliar dissertações

A lição está atualmente sendo pré-visualizada. ×

VISÃO GERAL

O vídeo a seguir mostra o processo de desenvolvimento dessa primeira parte do projeto. As explicações estão disponíveis nos demais capítulos desta aula. Cada capítulo explica uma parte do código, além de dar referências adicionais. Cada parte do código produzido está disponível no Github, em `branches` específicas. Assim, você pode acompanhar a produção do código passo a passo.

Você também pode fazer o **download do PDF** do material dessa aula.

Veja que tanto o Java quanto o Spring Boot lançam novas versões regularmente. Além disso, o Spring Initializr também está sempre em atualização. Por isso, não se assuste se você ver uma tela diferente da que está no vídeo. Também, procure sempre usar a versão mais atualizada do Java e do Spring Boot - o vídeo é apenas uma referência ;)



Avançar

TECNOLOGIAS

JAVA

Java é a principal tecnologia desse curso. Mas, em época de React, Python e nuvem, será que Java ainda vale a pena? Esse é o assunto deste vídeo.

Os dados do vídeo já são um pouco antigos, e talvez você esteja se perguntando se esse resultado persiste. Sim, ele ainda é verdade. Veja que o gráfico ao lado mostra os resultados de 2025. Apesar do Java ter mudado de posição, observe que ele permanece no top 5.



Feb 2025	Feb 2024	Change	Programming Language	Ratings	Change
1	1		Python	23.88%	+8.72%
2	3	^	C++	11.37%	+0.84%
3	4	^	Java	10.86%	+1.79%
4	2	v	C	9.94%	-1.14%
5	5		C#	4.12%	-3.41%

QUAL DISTRIBUIÇÃO USAR?

Desde que a OpenJDK passou a ser a referencia para a implementação do Java, surgiram várias distribuições do Java. Neste curso, você pode usar qualquer distribuição do Java, desde que seja igual ou acima da versão 21. Nos últimos anos, eu tenho usado a Amazon Corretto. Por isso, se você ainda não tem o Java instalado, ou está usando uma versão mais antiga, sugiro que visite o site da Amazon Corretto, faça o download e instale a versão 21.

MATERIAL COMPLEMENTAR

1. TIOBE index
2. StackOverflow Survey
3. CodinGame Survey

FERRAMENTAS

Além do Java, vamos usar várias outras ferramentas neste curso. Este vídeo aborda as três principais: (i) Spring Boot, o principal *framework*; (ii) Maven, a ferramenta de *build*; e (iii) Git/Github, a ferramenta de gerenciamento de versões.

A New Relic tem uma **versão mais atualizada** dessa pesquisa.

MATERIAL COMPLEMENTAR

- OutSystems Trends Survey
- Java Ecosystem Survey (Snyk)
- Spring Website
- Maven Website
- Git Website
- Github na StackOverflow Survey
- Github Website

IDEs

Você vai precisar de um ambiente de desenvolvimento integrado (IDE) se quiser ter produtividade no desenvolvimento. Você pode usar qualque IDE que tenha familiaridade. Contudo, eu uso o Visual Studio Code (VS Code) para gravar os vídeos e apresentar as explicações. O VS Code não é uma IDE, é um editor de texto. Mas é um editor de texto com esteróides. Ele é extensível por meio de plugins, muito fácil de usar e atende tudo que precisamos neste curso. Este vídeo discute as IDEs mais usadas e introduz o VS Code.

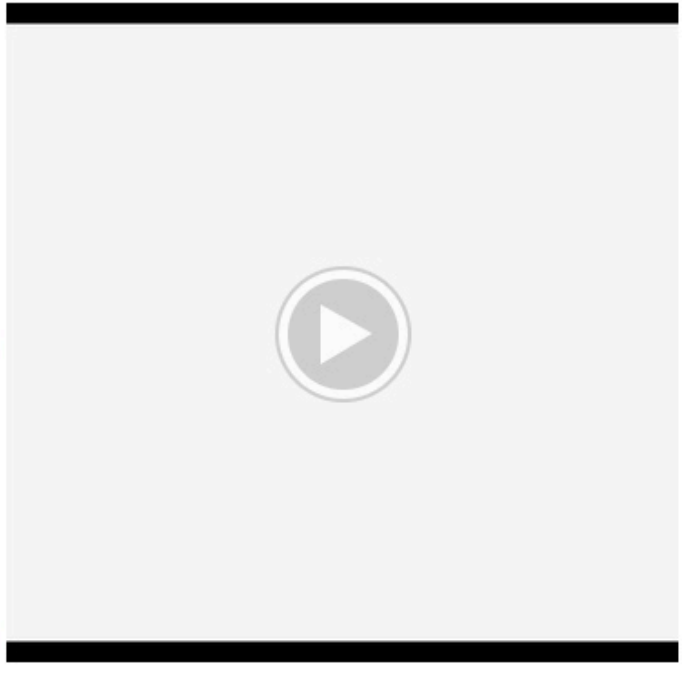
MATERIAL COMPLEMENTAR

- VS Code versão Java

TESTANDO O AMBIENTE

Chega de falar e vamos ver as coisas funcionando! Esse vídeo usa o `Petclinic`, uma aplicação de exemplo do Spring Boot, para testar um ambiente no qual as ferramentas de desenvolvimento que iremos usar são integradas.

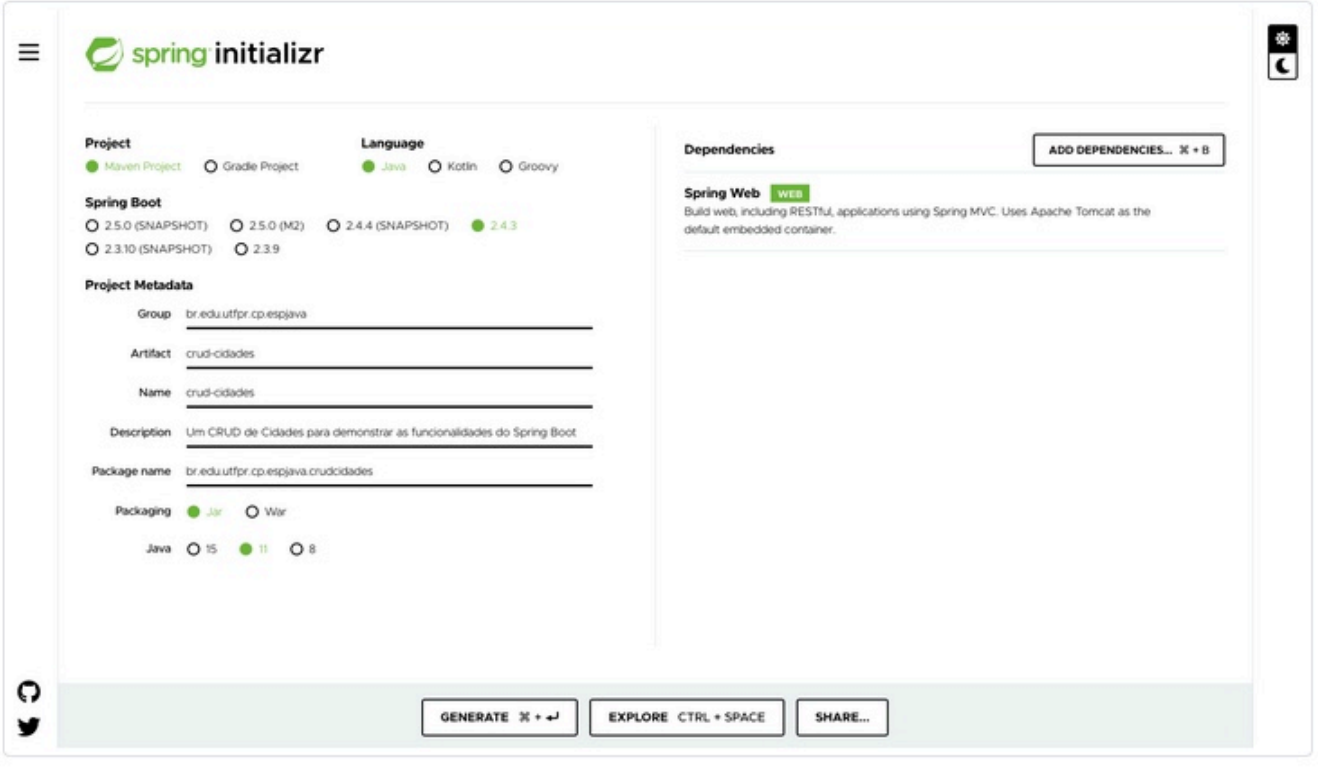
> Você pode usar esse projeto como referência durante a disciplina. Ele usa a maioria das tecnologias que usamos e pode ser uma excelente fonte de referência para conhecer outras técnicas.



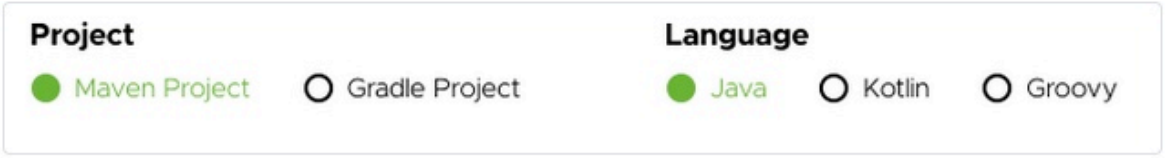
CRIANDO UM PROJETO SPRING BOOT

Este curso foi completamente atualizado para o Spring Boot 3.4.2 e Java 23.

O primeiro passo para criar um novo projeto usando o Spring Boot é visitar o Spring Initializr. O *Spring Initializr* permite a definição das características essenciais para o projeto, como a versão do Java que será usada e os dados do projeto. Além disso, ele permite selecionar as dependências quer serão necessárias no projeto.



Na parte superior do lado direito, a tela permite selecionar o tipo de projeto que será usado. Conforme definido na semana anterior, vamos usar o Maven para gerenciar as dependências do projeto. Do lado direito dessa opção, é possível escolher a linguagem que será usada. Apesar da JVM permitir outras linguagens além do Java, nessa disciplina nosso foco é Java.

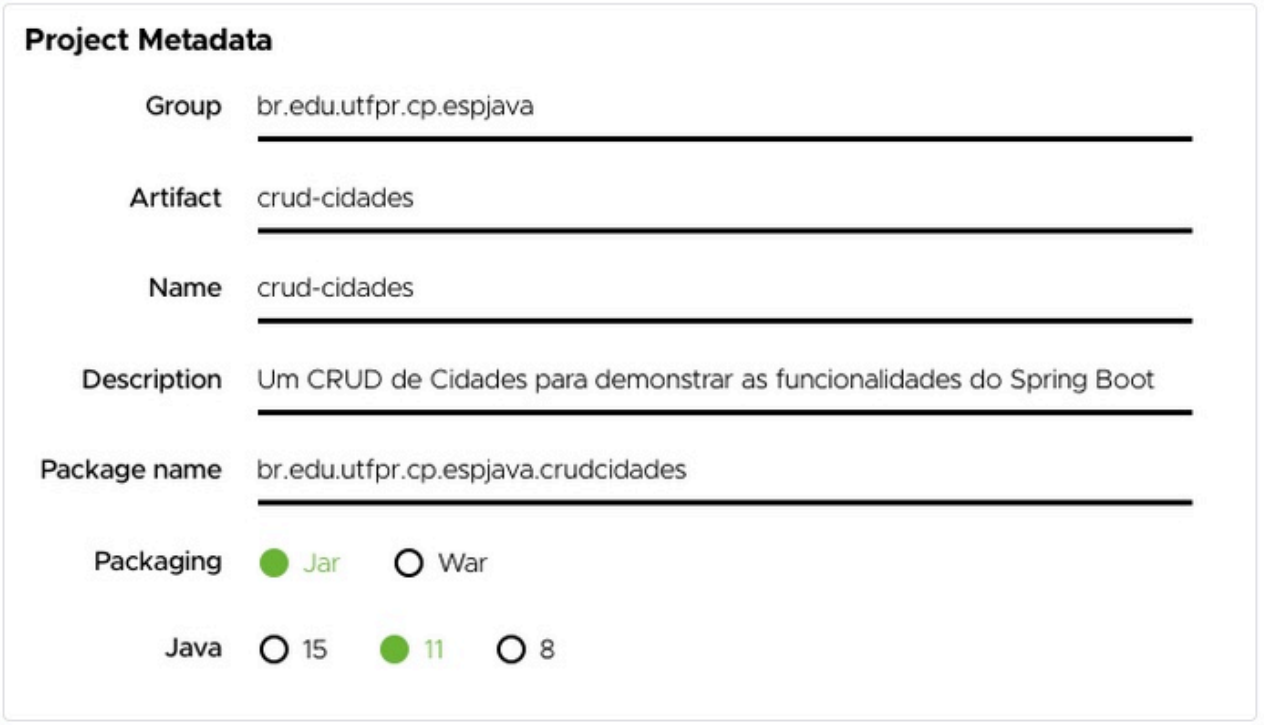


O Spring Boot é atualizado constantemente. Por isso, a figura abaixo é apenas uma ilustração da seleção de versões. Se tiver dúvidas, verifique a versão utilizada no código de exemplo no nosso repositório no Github.



Observe que o Spring Boot é atualizado constantemente. Por isso, o número da versão atual pode estar diferente do número da figura.

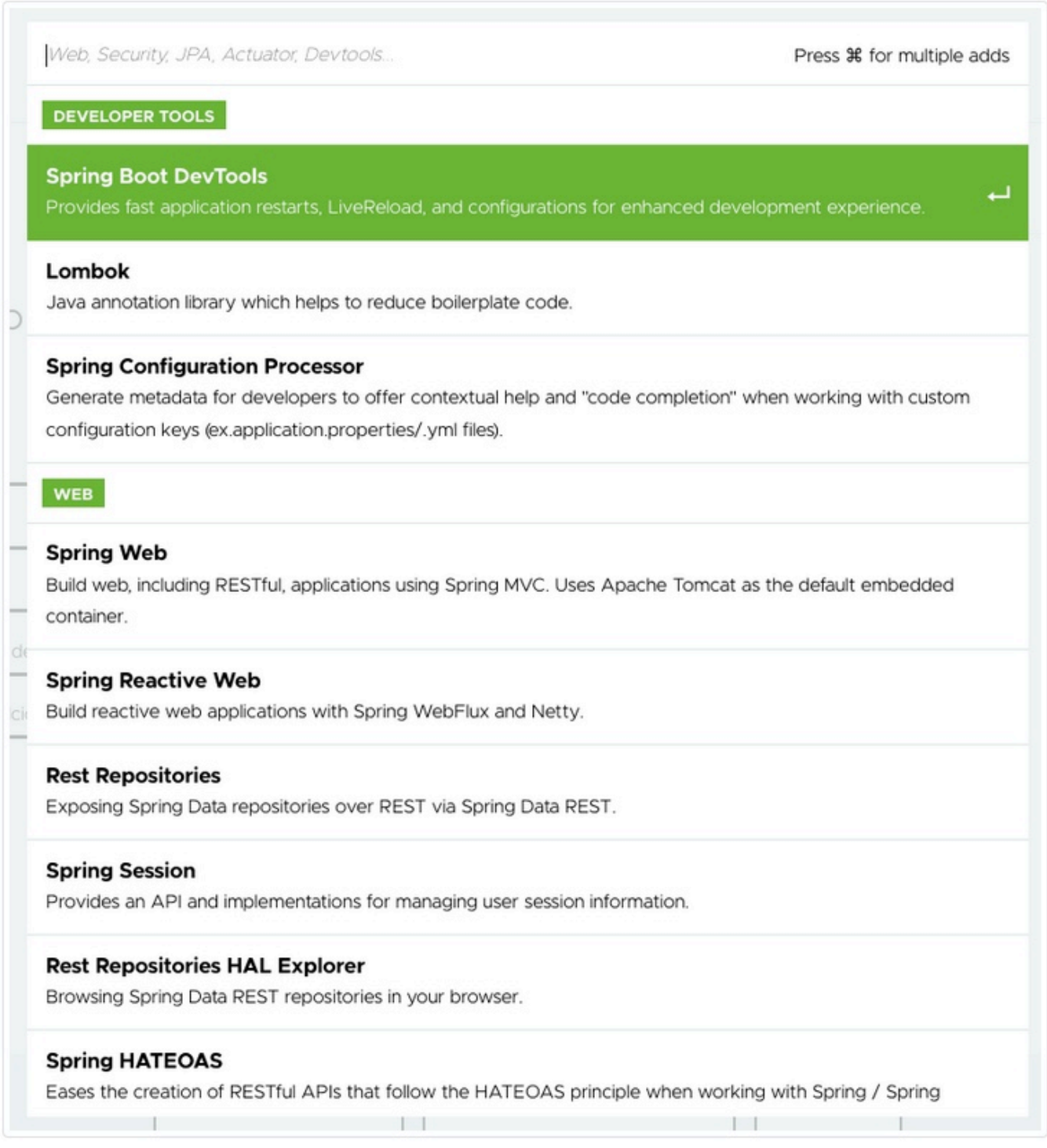
Em seguida, é preciso definir os meta-dados do projeto. Essas informações são usadas pelo Maven para, por exemplo, registrar o projeto no repositório do Maven. Também é preciso selecionar o tipo de empacotamento que será usado. No passado, aplicativos Web em Java costumavam ser empacotados em formato war. Contudo, hoje em dia o padrão é jar. Por isso, esse é o formato que iremos utilizar.



Um projeto de software consiste de uma ou mais dependências. Essas dependências são bibliotecas externas ou do próprio framework que são usadas para facilitar o desenvolvimento. O *Spring Initializr* permite a seleção de algumas dependências comuns em projetos com Spring Boot. Para selecionar as dependências necessárias para o projeto, basta clicar no botão `ADD DEPENDENCIES`, no lado direito da tela.



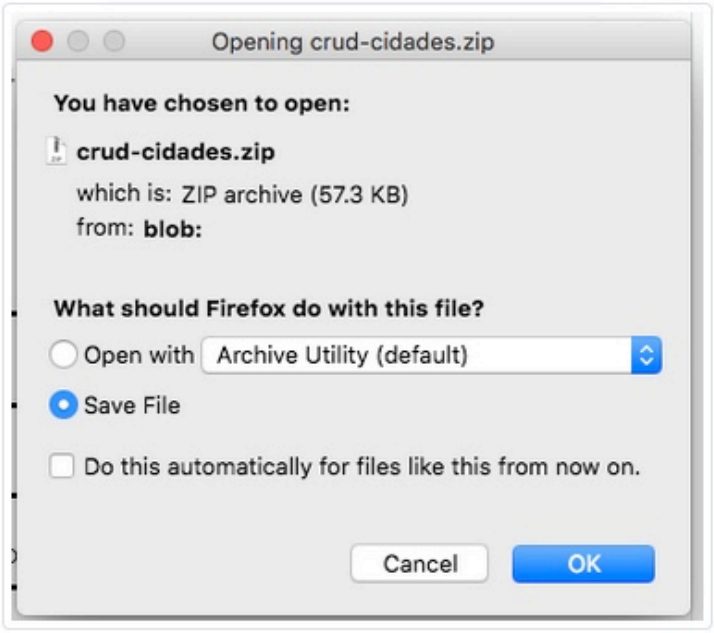
Na tela que é aberta, você pode selecionar suas dependências clicando sobre elas, selecionando com o teclado e pressionando o ENTER, ou informando o nome no campo de filtro. Para esse projeto, vamos usar várias dependências, mas vamos começar selecionando apenas a dependência `Spring Web`.



Para finalizar a criação do projeto, basta clicar no botão GENERATE, na parte inferior da tela. O *Spring Initializr* irá gerar um projeto Maven com as especificações informadas e dependências selecionas.



Em seguida, o navegador deve solicitar o download de um arquivo zip. Esse arquivo corresponde ao projeto comprimido. Salve o arquivo na pasta da sua preferencia. Em seguida, descomprima o arquivo e abra o projeto no seu editor preferido.



EXPLORANDO O PROJETO CRIADO

A Figura abaixo mostra a estrutura do projeto Maven criado pelo *Spring Initializr*. Na raiz, o arquivo mais importante é o `pom.xml`. É esse arquivo que armazena as configurações do Maven, o que inclui meta-dados e dependências, entre outros. Dentro da pasta `src`, existem duas pastas: `main` e `test`. Enquanto a pasta `main` é usada para os arquivos-fonte, a pasta `test` é usada para os arquivos de teste, como testes unitários, por exemplo.

```
.
├── HELP.md
├── LICENSE
├── mvnw
├── mvnw.cmd
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   └── br
│   │   │       └── edu
│   │   │           └── utfpr
│   │   │               └── cp
│   │   │                   └── espjava
│   │   │                       └── crudcidades
│   │   │                           └── CrudCidadesApplication.java
│   │   └── resources
│   │       ├── application.properties
│   │       ├── static
│   │       └── templates
│   └── test
│       └── java
│           └── br
│               └── edu
│                   └── utfpr
│                       └── cp
│                           └── espjava
│                               └── crudcidades
│                                   └── CrudCidadesApplicationTests.java
└── wrapper
    ├── MavenWrapperDownloader.java
    ├── maven-wrapper.jar
    └── maven-wrapper.properties
```

A pasta `main` se divide em java e `resources`. Enquanto a pasta `java` guarda as classes Java usadas no projeto, a pasta `resources` guarda os arquivos de configuração (`application.properties`), além de outras duas pastas. A pasta `static` é usada para armazenar conteúdo estático, como páginas `html` estáticas, enquanto a pasta `templates` é usada para armazenar conteúdo dinâmico, como páginas `html` que são alteradas dinamicamente durante a execução da aplicação.

Vocês vão perceber também que o projeto já vem com duas classes Java: uma dentro da pasta `main` e outra na pasta `test`. A classe Java dentro da pasta `main` contém o método que inicializa a aplicação, enquanto a classe Java dentro da pasta `test` é um teste unitário de exemplo.

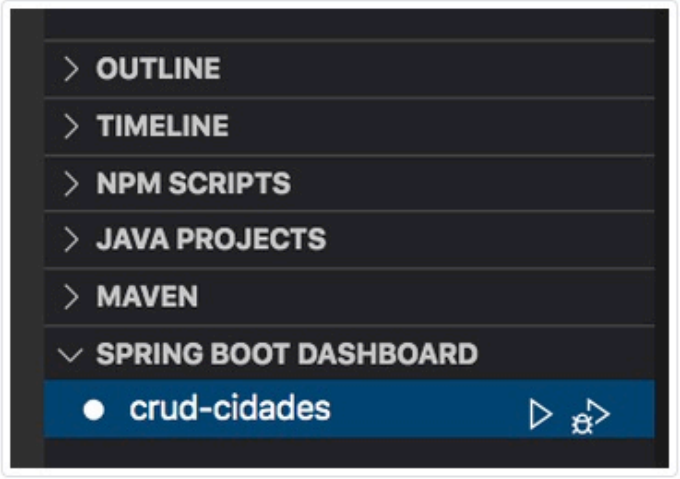
Com o projeto aberto no seu editor ou IDE de preferencia, vamos explorar a classe `CrudCidadesApplication.java`. Como vocês podem perceber, é uma classe Java comum. Ela tem o método `main`, que também é comum em aplicações Java. Esse método é primeiro método a ser executado quando a aplicação é inicializada.

O código dentro do método `main`, na linha 10, e anotação `@SpringBootApplication`, na linha 6, são usados pelo Spring Boot para definir que esse é um projeto Spring Boot e inicializar as configurações padrão para esse tipo de projeto.

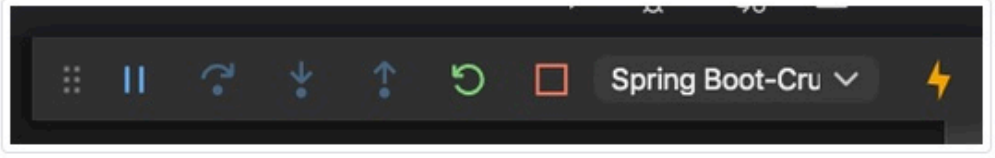
> Este projeto está disponível no **Github**, na branch `semana01-01-projeto-vazio`.

```
1   package br.edu.utfpr.cp.espjava.crudcidades;
2
3   import org.springframework.boot.SpringApplication;
4   import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6   @SpringBootApplication
7   public class CrudCidadesApplication {
8
        Run | Debug
9       public static void main(String[] args) {
10          SpringApplication.run(CrudCidadesApplication.class, args);
11      }
12
13  }
```

Se você está usando o VS Code com os plug-ins para Java e Spring Boot recomendados na aula anterior, você pode executar a aplicação clicando no botão `play`, no janela `SPRING BOOT DASHBOARD`.



Ao entrar em execução, você vai notar a janela de controle que, normalmente, é mostrada no canto superior direito da janela. O botão `STOP` (quadrado vermelho) é usado para parar a aplicação.



O painel inferior do editor mostra o log de execução do Spring Boot. A penúltima mensagem, `Tomcat started on port(s): 8080 (http) with context path ''`, mostra que a aplicação está em execução e pode ser acessada na porta 8080. Se você estiver usando uma IDE, como Netbeans ou Eclipse, o procedimento de execução não é muito diferente. Basta localizar o botão `play` e dar início à execução. Se você for um adepto da linha de comando, você pode inicializar o aplicativo entrando na pasta do projeto pelo seu terminal e digitando `mvn spring-boot:run`. Observe que você precisa ter seu ambiente configurado, com Java e Maven instalados.

É importante ressaltar que você deve usar a IDE ou editor que você tiver mais familiaridade para ter produtividade neste curso. Se você já tiver mais habilidade com tecnologia e quiser testar algo novo, então fique à vontade para se aventurar com o VS Code. Nada nesse curso é específico do VS Code, por isso, todo o conteúdo pode ser feito no editor/IDE da sua preferencia.



Na sua máquina local, basta abrir o navegador e digitar: `http://localhost:8080`. Contudo, observe que a aplicação ainda não faz nada. Por isso, tudo que você vai conseguir é uma mensagem de `erro 404`. Não se preocupe, agora vamos começar a moldar a aplicação da forma como queremos.



CRIANDO UMA PÁGINA ESTÁTICA COM BOOTSTRAP

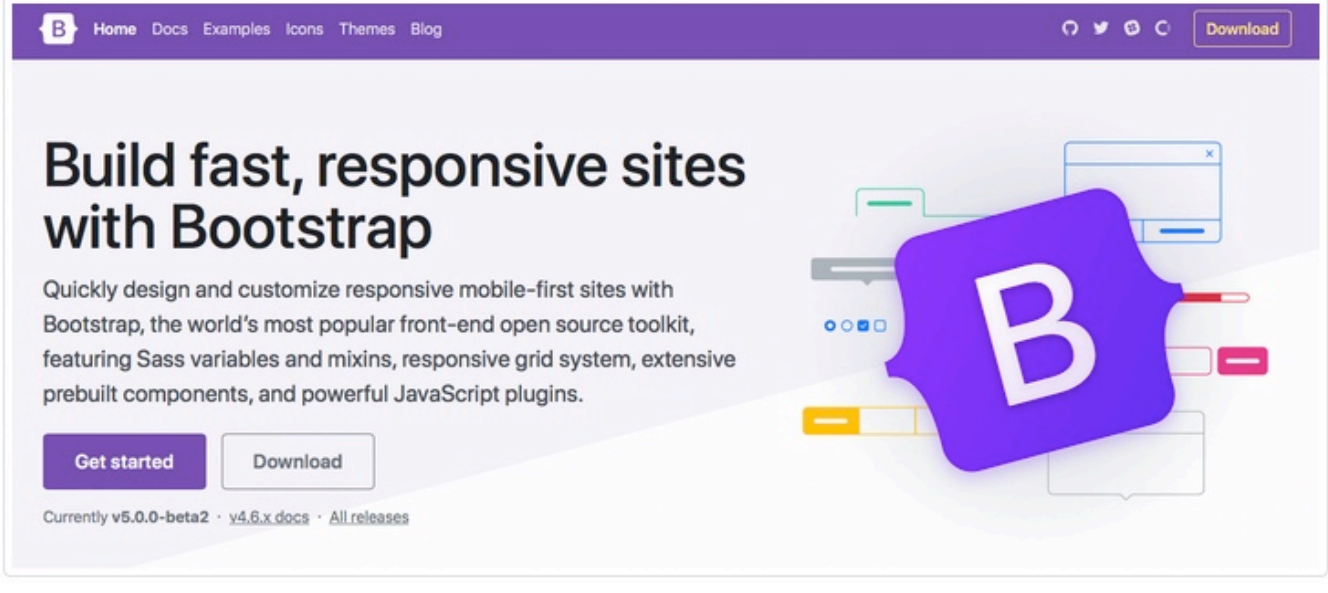
> Se o aplicativo ainda está em execução, pare a execução antes de continuar. Se o aplicativo foi iniciado em linha de comando, use `CTRL + C` para parar a execução.

Como primeiro projeto, vamos criar um CRUD de cidades. Um CRUD é a sigla usada para representar um aplicativo capaz de criar (*create*), ler (*read*), alterar (*update*) e excluir (*delete*) dados. Nesse caso, estamos usando a representação de uma cidade, que sempre pertence a um único estado. Esse exemplo é simples e familiar para todos. Para isso, precisamos de um protótipo de como será a página principal. Eu usei o `draw.io` para criar o protótipo na Figura a seguir.



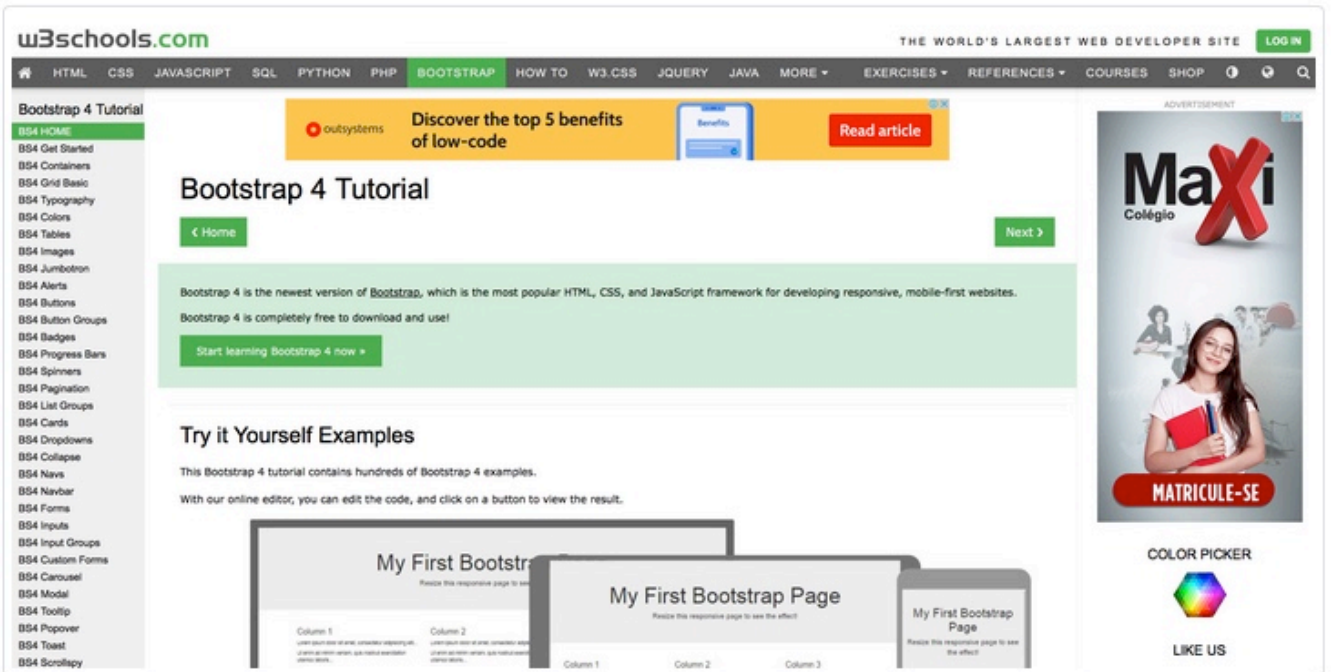
A página mostra um rótulo no topo, identificando o projeto. Logo em seguida, um formulário é usado para entrada de dados. Vamos usar apenas dois dados: nome e estado. É assim mesmo, esse primeiro projeto é para ser simples. Logo abaixo do formulário, vem uma tabela que lista as cidades já criadas. Para cada cidade listada na tabela, é possível alterar os dados, ou excluir a cidade, clicando no botão correspondente.

Para começar o projeto, crie o arquivo `crud.html` na pasta `/resources/static/`. Esse arquivo será usado para criar a página HTML que define nossa aplicação. Uma aplicação Web com Java consiste de HTML, CSS e, possivelmente, JavaScript, além de classes Java. Por isso, vamos sim usar HTML e CSS mesmo sendo uma aplicação Web Java. É claro que você pode criar todo seu CSS, formatando sua página da forma que achar melhor. Contudo, nesse curso nós vamos usar o Bootstrap. O Bootstrap é uma biblioteca que traz diversos esquemas de formatação já prontos. Com o Bootstrap você consegue formatar componentes HTML de forma rápida e simples.



No momento em que esse material está sendo atualizado, o Bootstrap está na versão 5.3.3. Para o nosso curso, a versão do Bootstrap é irrelevante. Por isso, você pode usar o Bootstrap 5 se sentir mais confortável. Apenas atente para as mudanças entre as versões. Por exemplo, o componente usado como título (`jumbotron`) não está mais disponível na versão 5.

Eu não sou um expert em CSS nem em Bootstrap. Sério, falo isso sem medo. Nós não precisamos saber tudo, mas precisamos conhecer quem sabe ;) Por isso, eu frequentemente uso a referencia do W3Schools, além da documentação do próprio Bootstrap quando preciso criar meus projetos. Lembrando que esse NÃO é um curso de Bootstrap. Por isso, vamos usar apenas o que precisamos para montar nossa página.



A primeira coisa que precisamos fazer é adicionar conteúdo HTML ao arquivo `/resources/static/crud.html`. Algo como a Figura abaixo.

```
1   <!DOCTYPE html>
2   <html lang="en">
3   <head>
4       <meta charset="UTF-8">
5       <meta http-equiv="X-UA-Compatible" content="IE=edge">
6       <meta name="viewport" content="width=device-width, initial-scale=1.0">
7       <title>CRUD Cidades</title>
8   </head>
9   <body>
10
11  </body>
12  </html>
```

Em seguida, precisamos tornar o Bootstrap disponível na nossa aplicação. Uma forma simples de fazer isso é usando CDN. O CDN é uma rede de servidores que dá acesso aos arquivos de configuração do Bootstrap. Basta adicionar o código da linha 9, conforme a Figura a seguir.

```
3   <head>
4       <meta charset="UTF-8">
5       <meta http-equiv="X-UA-Compatible" content="IE=edge">
6       <meta name="viewport" content="width=device-width, initial-scale=1.0">
7       <title>CRUD Cidades</title>
8
9       <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
10  </head>
```

Se você já tem mais experiência com Java e Spring Boot, talvez prefira usar **Webjars**.

O Bootstrap usa uma estrutura chamada `container` para definir o layout principal de uma página. Para isso, vamos criar uma seção principal usando um `div` e aplicando o formato `container-fluid`, conforme mostram as linhas 14-16, na Figura a seguir.

```
13  <body>
14      <div class="container-fluid">
15
16      </div>
17  </body>
```

Vamos usar um componente do Bootstrap chamado `jumbotron` para criar o rótulo superior que identifica o aplicativo, conforme mostra as linhas 16-19. Note que também adicionamos um espaço na margem superior (`mt-5`).

```
13  <body>
14
15      <div class="container-fluid">
16          <div class="jumbotron mt-5">
17              <h1>GERENCIAMENTO DE CIDADES</h1>
18              <p>UM CRUD PARA CRIAR, ALTERAR, EXCLUIR E LISTAR CIDADES</p>
19          </div>
20
21      </div>
22  </body>
```

Agora, o próximo passo é adicionar o formulário. Para isso, vamos criar um formulário logo abaixo do `jumbotron`.

```
<form>
    <div class="form-group">
        <label for="nome">Cidade:</label>
        <input type="text" class="form-control" placeholder="Informe o nome da cidade" id="nome">
    </div>
    <div class="form-group">
        <label for="estado">Estado:</label>
        <input type="text" class="form-control" placeholder="Informe o estado ao qual a cidade pertence"
            id="estado">
    </div>
    <button type="submit" class="btn btn-primary">CRIAR</button>
</form>
```

O último passo é adicionar uma tabela, logo abaixo do formulário. A tabela deve listar as cidades já existentes. Além disso, é possível alterar ou excluir uma cidade clicando nos botões correspondentes, apresentados na tabela. Observe que adicionamos vários componentes de formatação para tornar a tabela mais agradável.

```
32          <table class="table table-striped table-hover mt-5">
33              <thead class="thead-dark">
34                  <tr>
35                      <th>Nome</th>
36                      <th>Estado</th>
37                      <th>Ações</th>
38                  </tr>
39              </thead>
40              <tbody>
41                  <tr>
42                      <td>Londrina</td>
43                      <td>Paraná</td>
44                      <td>
45                          <div class="d-flex d-justify-content-center">
46                              <a class="btn btn-warning mr-3">ALTERAR</a>
47                              <a class="btn btn-danger">EXCLUIR</a>
48                          </div>
49                      </td>
50                  </tr>
51              </tbody>
52          </table>
```

A página está pronta e está bem similar ao protótipo definido. Você pode salvar o arquivo e abrir no seu navegador. Você também pode executar seu projeto e acessar a URL `http://localhost:8080/crud.html` direto no seu navegador.

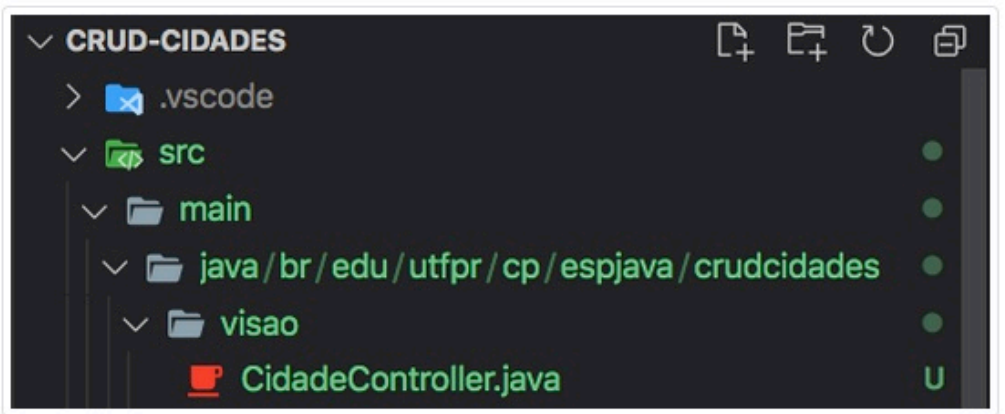
Este projeto está disponível no **Github**, na branch `semana01-02-projeto-pagina-estatica`.



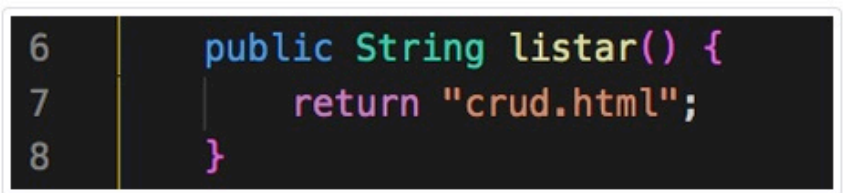
CRIANDO O ACESSO À PÁGINA

Agora que nossa página estática está pronta, precisamos criar uma classe Java que será responsável por receber as solicitações do navegador e enviar a página para o navegador. Para isso, vamos criar um pacote que vamos chamar de `visao`. No VS Code, basta criar uma pasta dentro da pasta `crudcidades`. Em seguida, crie a classe `CidadeController.java`, dentro da pasta `visao`. Se estiver usando outra IDE/editor, siga o procedimento adequado na sua IDE/editor para criar a classe o pacote.

> Um pacote em Java é representado na estrutura de diretórios como uma pasta.



Na classe `CidadeController`, crie um método que retorne o nome da página estática como uma `String`, conforme a Figura abaixo.

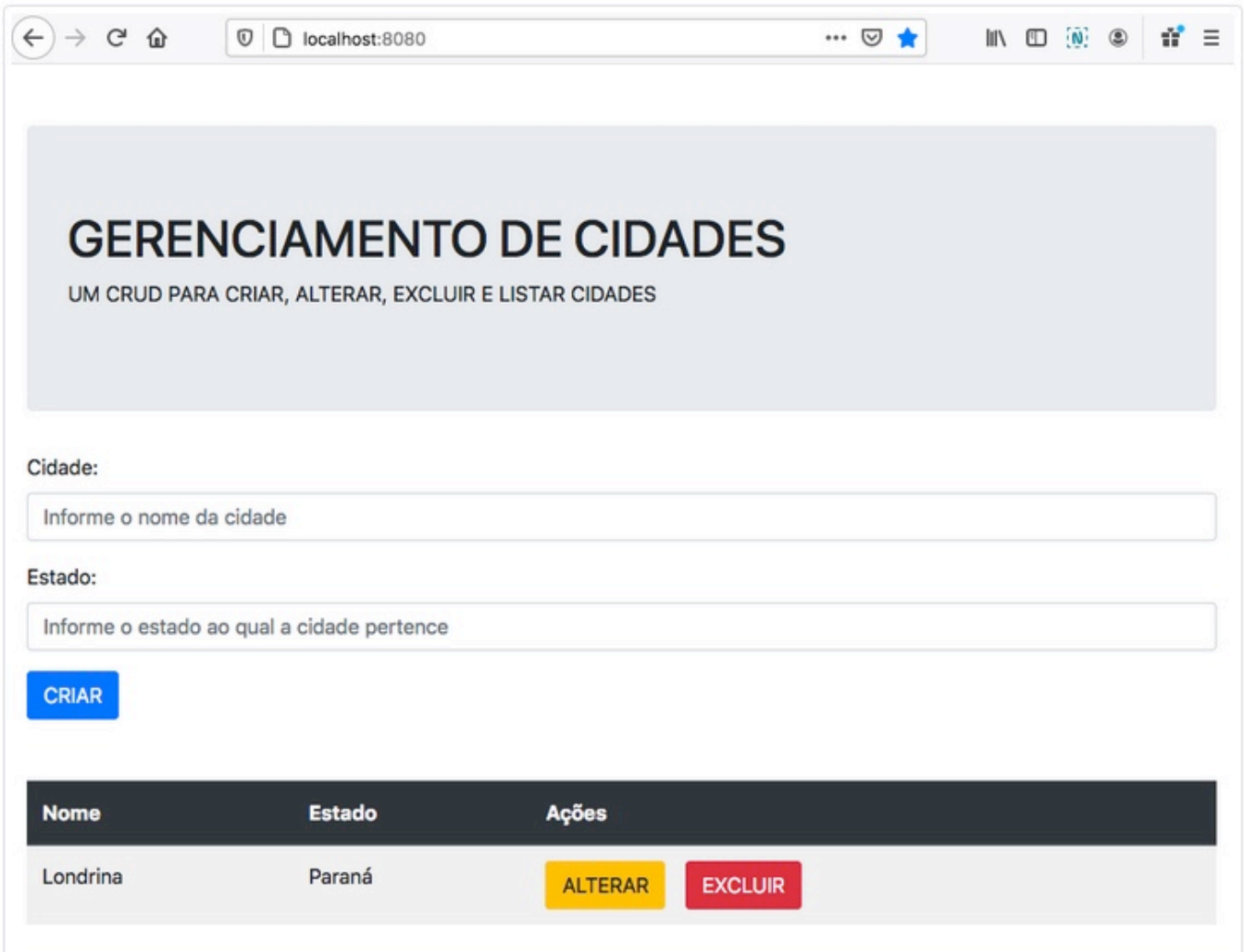


Por fim, adicione as anotações `org.springframework.stereotype.Controller` imediatamente antes da declaração da classe, e `org.springframework.web.bind.annotation.GetMapping` imediatamente antes da declaração do método.

> O VS Code também completa o código pra você, assim como as IDEs. Basta começar a escrever e ele já faz as sugestões. O VS Code também importa as classes usadas pelas bibliotecas referenciadas no `pom.xml`.

```
6   @Controller
7   public class CidadeController {
8
9       @GetMapping("/")
10      public String listar() {
11          return "crud.html";
12      }
```

A anotação na linha 6 indica pro Spring Boot que essa classe é uma classe que recebe requisições vindas do navegador. Já a anotação na linha 9 indica pro Spring Boot que qualquer solicitação feita para o endereço indicado, nesse caso, a raiz da URL, deve ser entregue para o método `listar()`. Quando o método retorna o nome da página estática, o Spring Boot entende que ele deve carregar essa página. Execute sua aplicação de dentro da sua IDE/editor ou usando `mvn spring-boot:run` em linha de comando. O resultado deve ser parecido com o da Figura a seguir.



O código desse projeto está disponível no **Github**, na branch `semana01-03-projeto-serve-estatica`.

Claro, essa página ainda não faz nada. Contudo, esses são os princípios fundamentais para criar uma aplicação Web com Java e Spring Boot. Agora, você pode comparar o código que você desenvolveu nessa aula com o código do `Petclinic`, apresentado na aula anterior. Na nossa próxima aula, vamos evoluir esse código adicionando dinamismo na página e criando um CRUD completo.

